

Rapport d'utilisation — SECONDAPP

Application web bancaire « premium » et module de facturation type Odoo

Auteur du projet : (à compléter)

Date du rapport : avril 2026

Emplacement du code : dossier `SECONDAPP` du dépôt `mobileapp-2026`

1. Contexte et fil du projet

1.1 Première itération et présentation

Le travail a débuté par un **premier projet** (application bancaire / mobile ou full-stack selon le périmètre initial) mené jusqu'à une **phase présentable** : démonstration fonctionnelle, soutenance ou rendu pédagogique. Cette première version a permis de valider des choix techniques, une première UX et une intégration avec un backend ou des données de démonstration.

1.2 Insatisfaction et décision de repartir de zéro

Après cette étape, le résultat **ne correspondait plus aux objectifs** (lisibilité du code, cohérence UI, modularité, alignement avec une maquette plus aboutie, ou simplicité de déploiement des données de test). Plutôt que d'empiler des correctifs sur une base jugée inadaptée, le choix a été fait de **tout recommencer depuis zéro** : nouvelle base front, nouvelle organisation des dossiers, et nouvelle stratégie pour les données fictives.

1.3 Naissance de la « deuxième application » — SECONDAPP

La **deuxième application** est le dossier `SECONDAPP` à la racine du monorepo. Elle n'est pas une simple copie : c'est une **refonte** orientée :

- **Interface** : expérience type application bancaire mobile « premium », à partir d'un bundle issu de l'écosystème Figma / Make (composants, thème, effets visuels).
- **Données** : API légère via **Docker + json-server** pour simuler un backend sans base SQL locale obligatoire.
- **Extension métier** : ajout d'un **module de facturation** calqué sur les concepts **Odoo** (`res.partner` , `account.move` , états brouillon / comptabilisé / paiement), avec édition des brouillons et création de factures côté client.

Le reste du dépôt peut contenir d'autres briques (ex. `project/firstapp` Flutter, `infrastructure/back-laravel` , etc.) ; ce rapport se concentre sur **SECONDAPP** et son usage.

2. Objectifs fonctionnels de SECONDAPP

Objectif	Description
Démonstration UX	Parcours bancaire crédible : accueil, solde, graphiques, transferts, crypto, investissements, cartes, analytics, sécurité, profil.
Données dynamiques	Charger graphiques et listes depuis un fichier JSON via une API HTTP locale.
Résilience	Si l'API n'est pas disponible, l'application reste utilisable avec un état « vide » ou message explicite.

Facturation	Consulter des factures et partenaires ; modifier les brouillons ; créer de nouveaux brouillons ; conserver les changements dans le navigateur (localStorage).
Présentation / portfolio	Projet lisible, documenté, reproductible sur une machine de développement Windows.

3. Architecture technique

3.1 Stack principale

- **Runtime / build** : Node.js, **Vite 6**, **TypeScript**.
- **UI** : **React 18**, **Tailwind CSS 4** (plugin Vite), composants type **shadcn/Radix** (Dialog, Sheet, Select, etc.).
- **Graphiques** : **Recharts** (courbes, aires) sur le tableau de bord et autres écrans.
- **Animation** : **Motion** (ex-Framer Motion).
- **Routing** : **React Router** (configuration dans `src/app/routes.tsx`).
- **Thème** : **ThemeContext** (dégradés, apparence clair/sombre selon implémentation).

3.2 Structure des dossiers (vue d'ensemble)

```

SECONDAPP /
├── docker/
│   └── db.json          # Données mock (dashboard + facturation imbriquée)
├── docker-compose.yml   # Service json-server, port hôte 3001
├── src/
│   ├── main.tsx
│   ├── app/
│   │   ├── App.tsx
│   │   ├── routes.tsx
│   │   ├── contexts/
│   │   ├── components/  # Écrans + UI + effets + modales
│   │   └── services/    # Appels API, persistance facturation, proxy URL
│   └── styles/
├── package.json
├── vite.config.ts       # Proxy /json-api → json-server
└── README.md

```

3.3 Flux de données (dashboard et facturation)

1. Le conteneur **Docker** expose **json-server** (image `clue/json-server`) sur `http://127.0.0.1:3001` (mapping `3001:80`).
2. Le fichier `docker/db.json` contient une clé racine **dashboard** avec :
 - `balanceData` , `transactions` , `insights` (tableau de bord) ;
 - **invoicing** : objets **partners** et **invoices** (facturation).
3. Le navigateur, en `npm run dev` , ne appelle pas directement le port 3001 pour éviter les problèmes **CORS** : Vite **proxifie** le chemin `/json-api` vers `http://127.0.0.1:3001` (voir `vite.config.ts` et `jsonServerBaseUrl.ts`).
4. Les services `dashboardApi.ts` et `invoicingApi.ts` consomment `GET /dashboard` (relatif au proxy, donc `/json-api/dashboard`).

3.4 Contrainte importante : json-server 0.12 (image clue/json-server)

L'image embarque **json-server en version 0.12**. Comportement observé : seule une ressource de type « objet racine » bien exposée fonctionnait de façon fiable pour l'usage choisi ; les routes `/partners` et `/invoices` en racine renvoyaient **404** alors que `/dashboard` répondait en 200.

Décision d'architecture : regrouper `partners` et `invoices` à l'intérieur de l'objet `dashboard`, sous la clé `invoicing`, pour tout charger via `GET /dashboard`. Cela simplifie le front et reste compatible avec cette version de json-server.

3.5 Persistance des brouillons de facturation

Le volume Docker monte `db.json` en **lecture seule** (`:ro`). Les écritures REST vers json-server ne mettraient pas à jour durablement le fichier sans changer le compose.

Choix produit : les **créations** et **modifications** de **brouillons** sont enregistrées dans `localStorage` (clé `secondapp-invoicing-drafts-v1`), avec **fusion** au chargement : les brouillons locaux **écrasent** les entrées de même `id` venant du JSON. Les factures **comptabilisées** / **payées** restent en **lecture seule** dans l'UI.

4. Parcours utilisateur par zone de l'application

4.1 Authentification (écrans Login / SignUp)

Écrans présents pour la cohérence du parcours « app bancaire » ; l'authentification réelle peut être simulée ou non branchée à un backend selon le contexte du cours ou du portfolio.

4.2 Layout principal

- **Barre du bas** : navigation entre Home, Transferts, Crypto, Investissements.
- **Zone flottante (haut droite)** : raccourcis vers Analytics, Cartes, Sécurité, Profil, et **Facturation** (icône reçu) vers `/invoices`.

4.3 Tableau de bord (/)

- Solde (affichage masquable), mini-graphique alimenté par `balanceData`.
- Liste des transactions et cartes d'« insights » si les données sont disponibles.
- Message explicite si l'API ne répond pas (Docker arrêté ou erreur réseau).

4.4 Autres écrans métier

- **Transferts** : envoi, demande, QR, fractionnement (UI + toasts).
- **Crypto / Investissements** : graphiques temps réel simulés ou données statiques selon l'écran.
- **Cartes, Analytics, Sécurité, Profil** : enrichissement UX du prototype.

4.5 Facturation (/invoices)

- **Liste** des factures avec filtres (Toutes, Brouillon, À encaisser, Payées).
- **Clic sur une facture** :
 - `state === draft` → ouverture de l'**éditeur** (modification autorisée tant que la facture n'est pas « envoyée » au sens métier : ici, tant qu'elle reste **brouillon**).
 - **Sinon** → **vue détail en lecture seule** (équivalent facture comptabilisée / payée).
- **Bouton « Nouvelle facture (brouillon) »** : création avec lignes éditables, client choisi dans la liste des partenaires, calcul automatique HT / TVA / TTC.
- **Enregistrement** : stockage local + message de confirmation.
- **Suppression** : réservée aux brouillons créés dans l'app (identifiants négatifs).

- **Reprise version serveur** : pour un brouillon issu du JSON (`id` positif), possibilité d'annuler les modifications locales.

5. Guide d'installation et d'utilisation (détailé)

5.1 Prérequis

- **Node.js** (LTS recommandé) et **npm**.
- **Docker Desktop** (Windows) pour lancer `docker compose`.
- Navigateur récent (Chrome, Edge, Firefox).

5.2 Installation

```
cd SECONDAPP
npm install
```

5.3 Lancer les données (Docker)

```
npm run docker:up
# équivalent : docker compose up -d
```

Vérification optionnelle : ouvrir ou interroger `http://127.0.0.1:3001/dashboard` (réponse JSON).

5.4 Lancer le front-end

```
npm run dev
```

Ouvrir l'URL indiquée par Vite (typiquement `http://localhost:5173`).

Important : après modification de `docker/db.json`, un `docker compose restart` peut être nécessaire pour que le conteneur relise le fichier selon l'image utilisée.

5.5 Variables d'environnement (optionnel)

- `VITE_API_BASE_URL` : URL directe du json-server ; à n'utiliser que si CORS est correctement configuré côté serveur. Par défaut, le projet utilise le **proxy** `/json-api`.

5.6 Build de production

```
npm run build
npm run preview
```

Le proxy est aussi défini pour `preview` ; un hébergement statique sans proxy nécessiterait une configuration serveur (Nginx, etc.) ou une URL d'API avec CORS.

6. Problèmes rencontrés et solutions retenues

Problème	Cause	Solution
----------	-------	----------

Données absentes malgré Docker	CORS : origine Vite ≠ origine port 3001	Proxy Vite /json-api + base URL relative
Facturation toujours vide	404 sur /partners et /invoices avec json-server 0.12	Données invoicing imbriquées sous dashboard ; un seul GET /dashboard
Impossible de « sauver » sur le disque	Volume db.json en lecture seule	Persistance localStorage pour brouillons
UX facturation	Besoin métier type Odoo	Modèle de champs aligné Odoo + états + lignes + totaux recalculés

7. Périmètre hors SECONDAPP (rappel)

Le dépôt peut contenir :

- **project/firstapp** : application Flutter MyBank branchée sur Laravel.
- **infrastructure/back-laravel** , **infrastructure/infra** , etc.

Ce rapport **ne remplace pas** la documentation de ces modules ; il documente **SECONDAPP** comme **deuxième application** dédiée au front web « premium » et à la facturation mock.

8. Perspectives d'évolution

- Brancher une **vraie API Odoo** (JSON-RPC / XML-RPC) ou un backend Laravel pour persister factures et partenaires.
- Mettre à jour l'image Docker vers un **json-server** récent ou un petit serveur **Express** custom avec CORS et routes REST explicites.
- Authentification réelle et cloisonnement des données par utilisateur.
- Tests automatisés (Vitest / Playwright) sur les flux dashboard et facturation.
- Internationalisation (i18n) si passage en production.

9. Glossaire

Terme	Signification
Brouillon	Facture state: draft — éditable dans SECONDAPP.
Comptabilisée	Facture state: posted — lecture seule dans l'UI actuelle.
json-server	Outil servant un fichier JSON comme fausse API REST.
Proxy Vite	Redirection des requêtes /json-api/* vers le conteneur local.
localStorage	Stockage navigateur pour les brouillons créés ou modifiés.

10. Conclusion

SECONDAPP matérialise une **second phase** du projet : refonte volontaire après une première présentation, avec une **UI riche**, des **données pilotées par Docker**, et un **module de facturation** pédagogiquement aligné sur **Odoo**, tout en contournant les limites techniques de l'outil mock (version json-server, CORS, écriture fichier) par des choix

d'architecture explicites. Ce document sert de **rapport d'utilisation** et de base pour un export PDF pour encadrement, portfolio ou soutenance.

Fin du rapport.